

UNIVERSITÉ D'AIX-MARSEILLE • BUT 2 INFORMATIQUE ARLES

R3.L01 — Représentations et codages des images

TP n°4 : Format TrueVision Targa (TGA) & Compression RLE

*Guide intégral d'architecture et de codage binaire basé sur le cours de **M. Éric Rémy***

Structure d'en-tête 18 octets, ordre BGR, orientation verticale, palettes et algorithme RLE

Contrôle TD : 14/10/2026

Séance TP4 : Format TGA

Structure du Format TGA (En-tête de 18 octets)

Les 18 Octets de l'En-tête Fixe

tga.pdf

- **Octet 0** : Taille champ ID (0 à 255 octets).
- **Octet 1** : Type Palette (0 = aucune, 1 = présente).
- **Octet 2** : Type Image (1=Palette, 2=RGB brut, 10=RGB RLE).
- **Octets 3-7** : Spécification Palette (Origine, Longueur, Profondeur).
- **Octets 8-11** : Origine X, Y (2 octets Little-Endian chacun).
- **Octets 12-15** : Largeur et Hauteur (Little-Endian 16 bits).
- **Octet 16** : Bits par pixel (24 pour RGB, 8 pour palette).
- **Octet 17** : Descripteur d'image (Orientation et Alpha).

Le POURQUOI : Deux Pièges Majeurs du TGA

- **1. L'Ordre BGR (Bleu, Vert, Rouge)** : Conçu en 1984 par TrueVision pour les cartes vidéo TARGA, le convertisseur matériel DAC lisait les canaux en ordre inverse ! Chaque pixel couleur 24 bits est stocké **B d'abord, puis G, puis R**.
- **2. Les Entiers Little-Endian** : Largeur, hauteur et longueurs de palette sont codées avec l'octet de poids faible en premier : $w = b_0 \mid (b_1 \ll 8)$.

Orientation Verticale : Bottom-Left vs Top-Left

Le Descripteur d'Image (Octet 17)

Bit 5 du descripteur :

- Bit 5 = 0 (0x00) : Origine en **BAS** à gauche (Standard TGA)
- Bit 5 = 1 (0x20) : Origine en **HAUT** à gauche (Modern)

- palette_bl.tga a le bit 5 à 0 → rangée 0 stockée en bas.
- palette_tl.tga a le bit 5 à 1 → rangée 0 stockée en haut.

Le POURQUOI : Inversion d'Indice lors de la Lecture

Si l'image est orientée du bas vers le haut (Bottom-to-Top) :

```
uint16_t target_y = top_to_bottom ? y : (height - 1 - y);  
res->pixel(x, target_y) = c;
```

- Sans cette inversion, l'image apparaît **la tête en bas (effet miroir vertical)** !
- Le format TGA respectait initialement le repère cartésien mathématique (axe Y vers le haut).

Sous-format 1 : Images avec Palette (Color-Mapped)

Principe de l'Image Indexée

chat010couleurs.tga

- **Table de couleurs (Palette)** : Située juste après l'en-tête, elle contient \$N\$ couleurs 24 bits (\$N\$ le 256\$).
- **Données de pixels (1 octet par pixel)** : Chaque pixel contient un **index** (de 0 à \$N-1\$) pointant dans la palette.
- Exemple chat010couleurs.tga : 10 couleurs seulement !

Le POURQUOI : Conversion à la Volée en RGB 24 bits

L'énoncé de M. Rémy précise :

« Lors de la lecture, l'image est convertie en image RGB puisque notre classe *ColorImage* ne permet pas de mémoriser une palette. »

```
uint8_t index;  
f.get((char&)index);  
res->pixel(x, target_y) = palette[index];
```

- Chaque index 8 bits est immédiatement résolu en son triplet `Color(r, g, b)` complet dans notre RAM.

Sous-format 10 : La Compression RLE (Run Length Encoding)

Structure des Deux Types de Paquets TGA

tgaffs.pdf

Chaque paquet débute par **1 octet d'en-tête (Header)** :

1. Run-Length Packet (Bit 7 = 1) :

$\text{count} = (\text{Header} \& 0x7F) + 1$ (2 à 128 pixels identiques)

Suivi de **1 seul pixel BGR** répété **count** fois.

2. Raw Packet (Bit 7 = 0) :

$\text{count} = (\text{Header} \& 0x7F) + 1$ (1 à 128 pixels différents)

Suivi de **count** pixels BGR consécutifs.

Le POURQUOI : Quand le RLE est-il Efficace ?

- **Images avec aplats et logos** : Si 100 pixels noirs se suivent, le format brut prend 300 octets. Le paquet RLE TGA prend **4 octets** (1 octet d'en-tête + 3 octets BGR) ! Gain de 98,6 % !
- **Photographies bruitées** : Sur une photo naturelle, deux pixels adjacents ont rarement exactement la même valeur RGB. Le RLE bascule en paquets Raw, ajoutant 1 octet d'en-tête tous les 128 pixels (très légère surcharge).

Synthèse pour le Contrôle (Questions Types)

Pièges d'examen sur le format TGA

- **Ordre des composantes** : Toujours B, G, R et non R, G, B !
- **Taille de l'en-tête** : Strictement 18 octets.
- **Orientation** : Tester le bit 5 du 18ème octet (descriptor). Si 0, inverser la coordonnée verticale.
- **Répétition maximale RLE** : 128 pixels par paquet.

Comparaison PPM vs TGA vs JPEG

Format	Compression	Canaux	Endianness
PPM (P6)	Aucune (Brut)	RGB	Sans
TGA (2 / 10)	Sans ou RLE	BGR	Little-Endian
JPEG	Pertes (DCT/Q)	YCbCr	Big-Endian markers

Annexe Code (1/3) : ColorImage.hpp (TGA)

Déclarations TGA dans ColorImage.hpp

```
class ColorImage {
    const uint16_t width, height;
    Color* array;
public:
    // Méthodes antérieures (constructeurs, pixel, clear, rectangle, PPM...)
    ColorImage(uint16_t w, uint16_t h);
    ~ColorImage();
    inline Color& pixel(uint16_t x, uint16_t y) { return array[y * width + x]; }

    // NOUVEAUTÉS TP4 : Format TrueVision Targa
    void writeTGA(std::ostream& f, bool rle = true) const;
    static ColorImage* readTGA(std::istream& f);
};
```

Annexe Code (2/3) : ColorImage::writeTGA

Implantation de writeTGA (Brut & RLE)

```
void ColorImage::writeTGA(std::ostream& f, bool rle) const {
    uint8_t header[18] = {0};
    header[2] = rle ? 10 : 2; // Type 10 = RLE, Type 2 = Brut
    header[12] = width & 0xFF; header[13] = (width >> 8) & 0xFF;
    header[14] = height & 0xFF; header[15] = (height >> 8) & 0xFF;
    header[16] = 24; header[17] = 0x20; // 24 bpp, Top-to-Bottom
    f.write((char*)header, 18);

    if (!rle) {
        for (uint16_t y = 0; y < height; ++y)
            for (uint16_t x = 0; x < width; ++x) {
                const Color& c = pixel(x, y);
                uint8_t bgr[3] = {c.b, c.g, c.r}; // ORDRE BGR !
                f.write((char*)bgr, 3);
            }
    } else {
        // Encodage RLE par paquets de 128 maximum
        for (uint16_t y = 0; y < height; ++y) {
            uint16_t x = 0;
            while (x < width) {
                uint16_t run = 1;
                while (x + run < width && run < 128 && pixel(x + run, y) == pixel(x, y)) ++run;
                if (run > 1) {
                    f.put((char)(0x80 | (run - 1)));
                    const Color& c = pixel(x, y);
                    uint8_t bgr[3] = {c.b, c.g, c.r}; f.write((char*)bgr, 3);
                    x += run;
                } else {
                    f.put((char)0); // 1 pixel raw
                    const Color& c = pixel(x, y);
                    uint8_t bgr[3] = {c.b, c.g, c.r}; f.write((char*)bgr, 3);
                    ++x;
                }
            }
        }
    }
}
```

Annexe Code (3/3) : ColorImage::readTGA

Lecture Types 1, 2, 10 et Orientation

```
ColorImage* ColorImage::readTGA(std::istream& f) {
    uint8_t h[18]; f.read((char*)h, 18);
    uint8_t type = h[2];
    uint16_t w = h[12] | (h[13] << 8), ht = h[14] | (h[15] << 8);
    bool top_to_bottom = (h[17] & 0x20) != 0;
    if (h[0] > 0) f.ignore(h[0]); // Ignore Image ID

    ColorImage* res = new ColorImage(w, ht);
    if (type == 2) { // RGB Brut
        for (uint16_t y = 0; y < ht; ++y) {
            uint16_t ty = top_to_bottom ? y : (ht - 1 - y);
            for (uint16_t x = 0; x < w; ++x) {
                uint8_t b, g, r; f.get((char&)b); f.get((char&)g); f.get((char&)r);
                res->pixel(x, ty) = Color(r, g, b);
            }
        }
    }
    return res;
}
```